

Quiz 3 – Applied OS: Cyber-Defense & Agentic AI

Q1		Total
(5)	(5)	(10)

L1: Student Name: _____

- Q1 Write a simple program where you first use `malloc()` to assign some memory to a pointer named `ptr`. (5)
a) Thereafter, you use `realloc()` to allocate a larger memory block to `ptr`.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *ptr;
    ptr = malloc (sizeof(int));

    // allocate a larger memory block

    int *tmp = realloc (ptr, 5 * sizeof(int));
    ptr = tmp;

    // Remember to free

    free(ptr);
    return 0;
}
```

- (b) Program 5.1: (5)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = malloc(sizeof(int));
    free(p);

    int *q = malloc(sizeof(int));
    *q = 100;
    printf("[q: %d]\n", *q);

    *p = 5;

    printf("[p: %d] \n", *p);
    printf("[q: %d] \n", *q);

    return 0;
}
```

Program 5.1 should have generated an error, but it doesn't; rather, it prints the following: but what is happening?

```
[q: 100]
[p: 5]
[q: 5]
```

1. The program exhibits undefined behavior from using a pointer after freeing it.
2. "p" is allocated with "malloc", then "free(p)" releases the memory, but "p" is not set to "NULL," becoming a dangling pointer.
3. When "q = malloc(sizeof(int))" is called next, the allocator often reuses the same just-freed memory block, so "p" and "q" may point to the same address. Therefore, "*q = 100" stores 100 and prints "[q: 100]".
4. Then "*p = 5" illegally writes through the freed pointer to that same memory, overwriting the value that "q" sees. So "*p" prints 5, and "*q" also prints 5, giving the output "[p: 5]" and "[q: 5]".
5. This is not guaranteed; it only happens because of allocator reuse, and the program could crash or behave differently elsewhere.